



FastAPI MASTERY SERIES

FastAPI

The Complete Hands-On Guide

From zero API knowledge to confidently building real, validated, self-documenting web APIs in Python.

● PHASE 1 · BEGINNER

Setup & Installation

GET · POST · PUT · DELETE

Path Parameters

Query Parameters

Request Body

Pydantic Models

Response Models & Status Codes

Forms & File Uploads

Swagger UI & ReDoc

Hands-on project for this phase: **The BookShelf API** · Built on **FastAPI 0.132+**, **Python 3.10+**, **Pydantic v2**

Verified against the official documentation at fastapi.tiangolo.com · 2026 Edition

How to use this guide

This is not a reference manual you skim. It is a workshop you follow with your hands on the keyboard. Every concept is introduced in plain language, justified with a reason, and immediately wired into one real project that grows with you.

Who this is for

This guide assumes you can write basic Python (variables, functions, lists, dictionaries) but knows *nothing* about web APIs. You have never seen an HTTP request, you are unsure what "REST" means, and the word "endpoint" sounds vague. That is exactly the right starting point. By the last page of Phase 1 you will be an **intermediate FastAPI developer** who can design, validate, document, and run a real API without copying from anyone.

How each topic is taught

Every topic follows the same rhythm so your brain can relax and learn:

1. **The idea in plain words** — what the thing is and the problem it solves.
2. **The mechanics** — the exact FastAPI syntax, explained line by line.
3. **Hands-on** — we add it to the running project and run it for real.
4. **Why it matters** — what would go wrong if you skipped it.

The boxes you will see

API TERM

Defines a word from the API world the first time it appears. If a term feels fuzzy, this box makes it concrete.

WHY IT MATTERS

Explains the practical payoff so you are never memorizing rules without understanding.

PRO TIP

A professional habit from real production codebases — small things that separate beginners from pros.

WATCH OUT

A common mistake or a gotcha that bites newcomers. Reading these saves you hours of debugging.

TRY IT YOURSELF

A short exercise. Do it before moving on — the knowledge only sticks when your fingers type it.

YOUR PHASE 1 PROJECT



The BookShelf API

A personal library manager. You will build it one feature at a time — listing books, fetching a single book, filtering and searching, adding new books with validation, returning clean responses, and even uploading cover images. Each topic adds one real capability, so by the end you hold a complete, working API you designed yourself.

All code in this guide is written for the modern FastAPI style used in the current official documentation: the `fastapi` command-line tool, Python type hints with `Annotated`, and Pydantic v2.

What you will learn

0 The API World — Core Concepts

Client/server, HTTP, REST, JSON, status codes

1 Setup & Installation

Virtual environments, installing FastAPI, your first app

2 Path Operations: GET, POST, PUT, DELETE

The four verbs that power every API

3 Path Parameters

Capturing values from the URL path, with type conversion

4 Query Parameters

Optional inputs for filtering, search and pagination

5 Request Body

Receiving structured JSON data from the client

6 Pydantic Models

Validation, types, fields and automatic error messages

7 Response Models & Status Codes

Controlling exactly what goes back to the client

8 Forms & File Uploads

Handling HTML forms and uploading book covers

9 Automatic Docs: Swagger UI & ReDoc

The free interactive documentation FastAPI builds for you

★ Capstone — The Complete BookShelf API

Every piece assembled into one runnable application

§ Glossary of API Terms

Every term from this phase in one place

→ What Comes Next — Phase 2 Preview

The API World — Core Concepts

Before a single line of FastAPI, you need a clear mental model of what an API actually is. Skip this and everything later feels like magic. Read it once and the rest of the guide will feel obvious.

What is an API?

Imagine a restaurant. You (the customer) never walk into the kitchen. You read a menu, tell a waiter what you want, and the waiter brings food back. You don't need to know how the kitchen works — you just need to know how to ask. An **API** is that waiter for software.

A API TERM — API (Application Programming Interface)

A well-defined way for one program to ask another program to do something or to hand over some data. A **web API** does this over the internet using the same technology your browser uses to load web pages.

When a weather app on your phone shows today's forecast, it didn't measure the temperature itself. It sent a request over the internet to a weather company's **web API**, which sent back the numbers. Your app is the **client**; the weather company runs the **server**. FastAPI is a tool for building that server side.

Client and server

Every interaction has two roles. The **client** asks; the **server** answers. A client can be a phone app, a website's JavaScript, another company's backend, or even a testing tool. The server is your FastAPI program, sitting and waiting for requests.

A API TERM — Request & Response

A **request** is the message a client sends ('please give me book #5'). A **response** is the message the server sends back (the book's data, plus a status code saying whether it worked). One request, one response. Always paired.

HTTP: the language they speak

HTTP (HyperText Transfer Protocol) is the agreed-upon format for these messages. It is the same protocol your browser uses — that is why web addresses start with `http://` or `https://`. An HTTP request carries four important things:

Part	What it is	Example
Method (verb)	The kind of action you want	<code>GET</code> , <code>POST</code>
Path (URL)	Which resource you are addressing	<code>/books/5</code>
Headers	Extra info about the request	<code>Content-Type: application/json</code>

Part	What it is	Example
Body	The data you are sending (not always present)	<code>{"title": "Dune"}</code>

The HTTP methods (verbs)

The same path can mean different things depending on the verb. `/books` with `GET` means "give me the books"; with `POST` it means "add a new book". These four verbs cover almost everything you will ever build, and they map onto a pattern so common it has a nickname — **CRUD**.

Verb	Intent	CRUD action	BookShelf example
<code>GET</code>	Read / fetch data	Read	Get the list of books, or one book
<code>POST</code>	Create something new	Create	Add a new book to the shelf
<code>PUT</code>	Replace / update	Update	Edit an existing book's details
<code>DELETE</code>	Remove something	Delete	Remove a book from the shelf

A API TERM — CRUD

Create, Read, Update, Delete — the four basic things you can do to stored data. Most APIs are, at heart, CRUD over some collection of things (books, users, orders, messages).

REST: the style FastAPI encourages

You will hear "REST API" constantly. **REST** is not a technology you install; it is a *style* of designing APIs that the industry settled on. Its core ideas are simple: treat your data as **resources** (a book, a user), give each resource a clear URL, and use the HTTP verbs above to act on them. FastAPI nudges you into RESTful design naturally.

A API TERM — Endpoint

A single combination of a **verb + path** that your API responds to — for example `GET /books`. Also called a **route** or, in FastAPI's own wording, a **path operation**. Your API is essentially a collection of endpoints.

JSON: how data travels

Computers need a shared text format to exchange structured data. The winner is **JSON** (JavaScript Object Notation). It looks almost exactly like a Python dictionary, which is part of why Python and APIs are such a comfortable pair.

A book represented as JSON

```
{
  "id": 5,
  "title": "Dune",
  "author": "Frank Herbert",
}
```

```

"year": 1965,
"in_stock": true,
"tags": ["sci-fi", "classic"]
}

```

Notice the building blocks: text in double quotes, numbers, `true` / `false`, lists in `[ ]`, and objects (key–value maps) in `{ }`. FastAPI converts between JSON and Python objects for you automatically — you almost never write JSON parsing code by hand.

Status codes: did it work?

Every response carries a three-digit **status code** that tells the client what happened, at a glance, before it even looks at the body. They are grouped by their first digit.

Range	Meaning	Common examples
2xx	Success	<code>200</code> OK · <code>201</code> Created · <code>204</code> No Content
3xx	Redirection	<code>301</code> Moved · <code>304</code> Not Modified
4xx	You (the client) made a mistake	<code>400</code> Bad Request · <code>401</code> Unauthorized · <code>404</code> Not Found · <code>422</code> Validation Error
5xx	The server broke	<code>500</code> Internal Server Error · <code>503</code> Service Unavailable

★ PRO TIP — Memorize three numbers to start

200 = success, **404** = not found, **422** = the client sent invalid data. FastAPI returns the right codes automatically in most cases, and you will learn to set them deliberately in Chapter 7.

Putting it together

Here is the whole picture in one sentence you can now fully understand: *a client sends an HTTP request (a verb + a path, sometimes with a JSON body) to your server; FastAPI matches it to an endpoint, runs your Python function, and sends back an HTTP response (a status code + usually a JSON body)*. Everything in this guide is just filling in the details of that loop.

❓ WHY IT MATTERS — This model is your foundation

FastAPI's job is to make this request → Python function → response loop effortless, safe, and self-documenting. Once you see every feature as 'this helps me read part of the request' or 'this helps me shape the response,' the entire framework clicks into place.

Setup & Installation

A clean, isolated environment is the difference between "it works on my machine" and a project you can trust. We will set one up properly, install FastAPI the recommended way, and run your very first endpoint.

Step 1 — Check your Python

FastAPI's current releases require **Python 3.10 or newer**. Check what you have:

```
python --version
# or, on some systems:
python3 --version
```

If it prints something like `Python 3.12.4`, you are ready. If it is older than 3.10, install a newer Python from python.org first.

Step 2 — Create a virtual environment

A **virtual environment** is a private, project-specific folder of installed packages. Without one, everything you install lands in your system-wide Python and projects start fighting over package versions. With one, each project is sealed off and reproducible.

A API TERM — Virtual environment (venv)

An isolated copy of Python plus its own package folder, living inside your project. Activating it means 'use these packages, not the global ones.' Professionals create one per project, every time, without exception.

Create a project folder, move into it, and make the environment:

```
mkdir bookshelf-api
cd bookshelf-api
python -m venv .venv
```

Now **activate** it. The command differs by operating system:

```
# macOS / Linux
source .venv/bin/activate

# Windows (PowerShell)
.venv\Scripts\Activate.ps1
```

Once active, your prompt usually shows `(.venv)` at the start. Everything you install now stays inside this folder.

★ PRO TIP — The .gitignore habit

Never commit the `.venv` folder to version control — it is large and machine-specific. Add a line containing `.venv/` to a `.gitignore` file. You recreate the environment from your dependency list, not from a copy.

Step 3 — Install FastAPI

The official recommendation is to install FastAPI with its **standard** extras. That single command brings along the pieces you need to actually run and develop an app — most importantly the **FastAPI command-line tool** and **Uvicorn**, the high-performance server that runs your code.

```
pip install "fastapi[standard]"
```

A API TERM — Uvicorn & ASGI

Your Python code does not talk to the network directly. A **server** like Uvicorn listens for HTTP requests and hands them to your app through a standard called **ASGI** (Asynchronous Server Gateway Interface). FastAPI is an ASGI framework, which is what lets it handle many requests concurrently and very fast.

Why the quotes around `"fastapi[standard]"`? Some terminals treat square brackets specially; the quotes make the command behave identically everywhere. The `[standard]` part pulls in Uvicorn, the `fastapi` CLI, form/file support, and more, so you are not hunting for missing packages later.

Confirm it installed:

```
python -c "import fastapi; print(fastapi.__version__)"  
# e.g. 0.132.0
```

Step 4 — Your first FastAPI app

Create a file called `main.py` with these few lines. This is a complete, working API — every word matters, and we will dissect it next.

```
main.py  
  
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/")  
def read_root():  
    return {"message": "Welcome to the BookShelf API"}
```

Line by line

- `from fastapi import FastAPI` — bring in the main class the framework gives you.

- `app = FastAPI()` — create your application object. The whole API hangs off this single `app`. The variable name `app` is a convention the tooling expects.
- `@app.get("/")` — a **decorator** that says "when a `GET` request arrives for the path `/`, run the function below." This pairing of verb + path is your first endpoint.
- `def read_root():` — the function that handles the request. Its name is for you; FastAPI never shows it to clients.
- `return {...}` — return a plain Python dictionary. FastAPI automatically converts it to JSON and sends it as the response body, with status `200`. You never touch JSON by hand.

A API TERM — Decorator & path operation

A **decorator** is the `@something` line attached above a function; it adds behaviour to that function. FastAPI calls a decorated handler a **path operation** — 'path' (`/`) plus 'operation' (`GET`). It is the same thing as an endpoint or route.

Step 5 — Run it

The `fastapi` command-line tool (installed with the standard extras) runs your app in development mode with automatic reloading:

```
fastapi dev main.py
```

You will see output telling you the server is running, typically at `http://127.0.0.1:8000`. That address means "this computer (`127.0.0.1`, also called *localhost*), port `8000`."

A API TERM — localhost & port

localhost / `127.0.0.1` always means 'this same computer.' A **port** is a numbered door on that computer; web servers commonly use 8000 in development. Together they form the address clients connect to.

Open a browser to `http://127.0.0.1:8000` and you will see your JSON message. You just served your first API response.

★ PRO TIP — dev vs run

`fastapi dev` enables auto-reload (the server restarts when you save a file) — perfect while building. For production you use `fastapi run`, which does not reload and is tuned for performance. We cover real deployment in Phase 4.

Step 6 — See the free documentation

Without writing one extra line, FastAPI has already built interactive documentation for your API. Visit:

- `http://127.0.0.1:8000/docs` — Swagger UI, where you can call your endpoints from the browser.
- `http://127.0.0.1:8000/redoc` — ReDoc, a clean reading-focused view.

We will return to these in Chapter 9 and lean on them constantly along the way.

 TRY IT YOURSELF — Make it yours

Change the returned message, save the file, and refresh the browser. Because `fastapi dev` auto-reloads, your change appears instantly. Then add a second endpoint at `/health` that returns `{{"status": "ok"}}` and visit it. You now have a two-endpoint API.

Path Operations: GET, POST, PUT, DELETE

The four HTTP verbs are the verbs of your whole API. Here you will declare all four on the BookShelf project and understand precisely when to reach for each one.

One decorator per verb

FastAPI gives you a decorator method for every HTTP verb, all hanging off your `app` object: `@app.get`, `@app.post`, `@app.put`, `@app.delete` (also `.patch`, `.head`, `.options`). You pick the decorator that matches the *intent* of the action.

? WHY IT MATTERS — Verbs carry meaning

Using the right verb is not decoration — it is communication. Clients, browsers, caches and other developers all assume `GET` only reads and never changes data, and that `DELETE` removes. Respecting these conventions makes your API predictable and safe to build on.

Building the BookShelf skeleton

We will store books in a simple Python list for now — a stand-in "database" that lives in memory. (In Phase 2 we replace it with a real database; the endpoint shapes stay the same.) Replace your `main.py` with this.

```
main.py

from fastapi import FastAPI

app = FastAPI(title="BookShelf API")

# A temporary in-memory "database": a list of dictionaries.
books = [
    {"id": 1, "title": "Dune", "author": "Frank Herbert"},
    {"id": 2, "title": "1984", "author": "George Orwell"},
]

@app.get("/books")          # READ the whole collection
def list_books():
    return books

@app.post("/books")         # CREATE a new book
def create_book(book: dict):
    book["id"] = len(books) + 1
    books.append(book)
    return book
```

```

@app.put("/books/{book_id}") # UPDATE one existing book
def update_book(book_id: int, book: dict):
    for i, existing in enumerate(books):
        if existing["id"] == book_id:
            book["id"] = book_id
            books[i] = book
            return book
    return {"error": "Book not found"}

@app.delete("/books/{book_id}") # DELETE one book
def delete_book(book_id: int):
    for i, existing in enumerate(books):
        if existing["id"] == book_id:
            books.pop(i)
            return {"deleted": book_id}
    return {"error": "Book not found"}

```

i NOTE — We will refine this

Right now `book: dict` accepts any data at all, and our 'not found' case returns a fake-success body. That is intentional — these are exactly the weaknesses Pydantic models (Ch. 6), response models and status codes (Ch. 7) exist to fix. Build it rough first, then tighten.

Reading the four endpoints

Endpoint	Verb + Path	What it does
List books	<code>GET /books</code>	Returns the full list
Create book	<code>POST /books</code>	Adds a new book sent in the request body
Update book	<code>PUT /books/{book_id}</code>	Replaces the book with that id
Delete book	<code>DELETE /books/{book_id}</code>	Removes the book with that id

Notice the design pattern, which is pure REST: the *collection* lives at `/books`, and a *single item* lives at `/books/{book_id}`. The verb decides whether you are reading, creating, updating or deleting. That same shape repeats for users, orders, anything.

POST vs PUT — the classic confusion

	POST	PUT
Purpose	Create something new	Replace an existing item
Target	The collection (<code>/books</code>)	A specific item (<code>/books/5</code>)
Run it twice	Creates two items	Same result as running once

	POST	PUT
Client sets the id?	No — server assigns it	Yes — id is in the path

API TERM — Idempotency

An operation is **idempotent** if doing it many times has the same effect as doing it once. `PUT` and `DELETE` are idempotent; `POST` is not (it keeps creating). Knowing this helps you pick the right verb and reason about retries.

Try every verb without leaving the browser

Run `fastapi dev main.py` and open `http://127.0.0.1:8000/docs`. Each endpoint is listed with a coloured button by verb. Expand `POST /books`, click *Try it out*, edit the example body, and press *Execute*. You will see the live response and the status code — no extra tools, no command line.

TRY IT YOURSELF — Exercise the full CRUD loop

From the `/docs` page: (1) `POST` a new book `{{"title": "Hyperion", "author": "Dan Simmons"}}`; (2) `GET /books` and confirm it appears with an id; (3) `PUT` to change its title; (4) `DELETE` it. You just performed every CRUD action against your own API.

Path Parameters

A path parameter is a value baked into the URL itself — the `5` in `/books/5`. FastAPI captures it, converts it to the right Python type, and hands it to your function as an argument.

Declaring a path parameter

You declare a variable part of the path by wrapping a name in braces, then accept a function argument with the same name:

```
@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"requested_id": book_id}
```

A request to `/books/5` calls `get_book` with `book_id == 5`. The brace name and the argument name must match exactly.

A API TERM — Path parameter

A piece of data passed as part of the URL path, used to identify *which* resource you want. It is required by definition — without it, the path simply does not match. Contrast with query parameters (next chapter), which are optional add-ons.

The magic of the type hint

Look again at `book_id: int`. URLs are always text, so the raw value arriving is the string `"5"`. Because you annotated it as `int`, FastAPI does two things for free:

1. **Converts** the text `"5"` into the integer `5` before your function runs.
2. **Validates** it. If a client requests `/books/banana`, FastAPI never even calls your function — it returns a clear `422` error explaining that an integer was expected.

Automatic 422 response for `/books/banana`

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "book_id"],
      "msg": "Input should be a valid integer, ...",
      "input": "banana"
    }
  ]
}
```

? WHY IT MATTERS — Type hints are your validation layer

In most frameworks you would hand-write code to parse and check every input. In FastAPI the ordinary Python type hint *is* the specification. Less code, fewer bugs, and the error messages are written for you. This single idea is the heart of FastAPI's appeal.

Wiring it into BookShelf

Add a "get one book" endpoint that actually searches the list:

```
@app.get("/books/{book_id}")
def get_book(book_id: int):
    for book in books:
        if book["id"] == book_id:
            return book
    return {"error": "Book not found"}  # we will improve this in Ch. 7
```

Order matters: fixed paths before variable ones

FastAPI matches routes top to bottom. A fixed path like `/books/featured` must be declared *before* the variable `/books/{book_id}`, or the word "featured" would be captured as a `book_id` and fail conversion.

```
@app.get("/books/featured")  # define the specific route FIRST
def featured_books():
    return books[:3]

@app.get("/books/{book_id}")  # the catch-all variable route SECOND
def get_book(book_id: int):
    ...
```

! WATCH OUT — A subtle ordering bug

If you put `/books/{book_id}` first, a request to `/books/featured` tries to turn 'featured' into an int and returns 422. The fix is always: **specific routes first, parameterized routes last.**

Restricting values with Enums

Sometimes a path value should only be one of a fixed set. A Python `Enum` expresses that, and FastAPI both validates against it and lists the allowed values in the docs.

```
from enum import Enum

class Genre(str, Enum):
    fiction = "fiction"
    science = "science"
    history = "history"
```



```
@app.get("/genres/{genre}")
def books_by_genre(genre: Genre):
    return {"genre": genre, "note": f"Listing all {genre.value} books"}
```

Now `/genres/fiction` works, while `/genres/poetry` is rejected automatically — and the docs show a dropdown of valid choices.

 TRY IT YOURSELF — Add featured + a single-book lookup

Add both the `/books/featured` and `/books/{book_id}` endpoints in the correct order. In `/docs`, fetch book `1`, then deliberately request `/books/abc` and read the 422 error FastAPI generated for you.

Query Parameters

Query parameters are the optional knobs after the `?` in a URL. They are how clients filter, search, sort and paginate — without you creating a new endpoint for every combination.

What they look like

In `/books?genre=sci-fi&limit=10`, everything after the `?` is a query string: pairs of `key=value` joined by `&`. Here `genre` is "sci-fi" and `limit` is "10".

A API TERM — Query parameter

An optional, named input attached to the end of the URL after `?`. Unlike path parameters (which identify *which* resource), query parameters usually *modify* or *filter* a request. They are perfect for things that are optional.

Declaring them is just adding arguments

Here is the elegant part: any function argument that is **not** part of the path is automatically treated as a query parameter. Give it a default value to make it optional.

```
@app.get("/books")
def list_books(skip: int = 0, limit: int = 10):
    return books[skip : skip + limit]
```

Now `/books` returns the first 10, while `/books?skip=10&limit=5` returns books 11–15. The type hints again give you free conversion and validation: `limit` must be an integer.

A API TERM — Pagination

Returning a large list in small chunks ('pages') instead of all at once. The `skip / limit` pair above is the simplest, most common pagination scheme. It keeps responses fast and payloads small.

Required vs optional query parameters

The default value decides everything:

Declaration	Behaviour
<code>limit: int = 10</code>	Optional, defaults to 10 if absent
<code>limit: int None = None</code>	Optional, may genuinely be missing (None)
<code>q: str</code>	Required — request fails with 422 if missing

```

from typing import Optional

@app.get("/books")
def list_books(
    skip: int = 0,
    limit: int = 10,
    author: str | None = None,  # optional filter; modern 3.10+ syntax
):
    results = books
    if author is not None:
        results = [b for b in results if b["author"] == author]
    return results[skip : skip + limit]

```

The `str | None = None` annotation says "a string, or nothing." When the client omits `author`, you skip the filter. This is how optional filtering is expressed cleanly.

Booleans are parsed intelligently

Annotate a parameter as `bool` and FastAPI accepts a range of truthy text forms and converts them for you:

```

@app.get("/books")
def list_books(available_only: bool = False):
    ...

# /books?available_only=true    -> True
# /books?available_only=1      -> True
# /books?available_only=no     -> False

```

Validating query parameters with Query()

For real APIs you want guard rails: a search term of a sensible length, a `limit` that cannot be set to a million. FastAPI provides `Query`, used together with `Annotated` — the modern, recommended way to attach rules to a parameter.

```

from typing import Annotated
from fastapi import Query

@app.get("/books/search")
def search_books(
    q: Annotated[str, Query(min_length=2, max_length=50)],
    limit: Annotated[int, Query(ge=1, le=100)] = 10,
):
    matches = [b for b in books if q.lower() in b["title"].lower()]
    return matches[:limit]

```

A API TERM — Annotated

`Annotated[Type, metadata]` is standard Python for attaching extra information to a type. FastAPI reads that metadata to learn the validation rules. It is the current best-practice style throughout the official docs because it keeps the default value clearly separate from the rules.

The validators read naturally: `ge` = greater-than-or-equal, `le` = less-than-or-equal, plus `min_length` / `max_length` for strings. Break a rule and the client gets a precise 422 explaining exactly which parameter failed and why.

? WHY IT MATTERS — Validate at the door

Catching bad input at the edge of your API — before it reaches your logic or database — is a core professional habit. It turns vague downstream crashes into clear, immediate error messages, and it is essentially free with FastAPI.

▶ TRY IT YOURSELF — Add search + filtering to BookShelf

Add the `/books/search` endpoint above and the `author` filter to `/books`. In `/docs`, search for `une` (should match 'Dune'), then try a one-character query and watch the validation reject it.

Request Body

When a client needs to send you a meaningful chunk of data — a new book, a user profile, an order — it goes in the **request body** as JSON. This is the input side of `POST` and `PUT`.

Path/query vs body — three places data can live

Where	Typical use	Size	Example
Path parameter	Identify the resource	Tiny	<code>/books/5</code>
Query parameter	Optional filters/options	Small	<code>?limit=10</code>
Request body	The actual data payload	Can be large	<code>{"title": "Dune", ...}</code>

A API TERM — Request body

The block of data a client sends in the body of an HTTP request, almost always as JSON. You use it whenever the client is *submitting* structured information, not just identifying or filtering. `GET` requests conventionally have no body.

The naïve way (and why we move past it)

You already saw `book: dict` in Chapter 2. It works, but it accepts *anything* — an empty object, a misspelled key, a title that is actually a number. You get no validation, no documentation, and no editor autocompletion.

```
@app.post("/books")
def create_book(book: dict):  # accepts literally any JSON object
    ...
```

! WATCH OUT — Untyped bodies are a trap

With `dict`, a client could POST `{"title": 42}` and your code happily stores garbage, then crashes later when it expects a real title. The fix is a **Pydantic model** — the entire subject of the next chapter — which is the proper way to declare a request body.

The right way: a typed body with a Pydantic model

We will go deep on Pydantic in Chapter 6, but here is the essential mechanism so request bodies make sense now. You describe the expected shape as a class, then use it as the type hint.

```
from pydantic import BaseModel

class BookIn(BaseModel):
```

```

title: str
author: str
year: int

@app.post("/books")
def create_book(book: BookIn):
    new = book.model_dump()      # turn the model into a plain dict
    new["id"] = len(books) + 1
    books.append(new)
    return new

```

Because the function argument is typed as a Pydantic model, FastAPI now:

- Reads the JSON body automatically — no manual parsing.
- Validates every field: `year` must be an integer, `title` must be present, and so on.
- Rejects bad payloads with a detailed `422` before your code runs.
- Hands you a fully-typed `book` object with editor autocompletion.
- Documents the expected JSON shape in `/docs` automatically.

A API TERM — `model_dump()`

A Pydantic v2 method that converts a model instance into an ordinary Python dictionary. (In the old Pydantic v1 it was called `.dict()`.) The reverse — building a model from a dict — is `BookIn(**data)`.

Body + path + query together

FastAPI figures out where each argument comes from by its kind, so you can freely combine all three in one function. This is exactly what a real `PUT` looks like:

```

@app.put("/books/{book_id}")
def replace_book(
    book_id: int,          # from the PATH
    book: BookIn,          # from the BODY (it is a Pydantic model)
    notify: bool = False,  # from the QUERY string (?notify=true)
):
    for i, existing in enumerate(books):
        if existing["id"] == book_id:
            updated = book.model_dump()
            updated["id"] = book_id
            books[i] = updated
            return {"updated": updated, "notified": notify}
    return {"error": "Book not found"}

```

i NOTE — How FastAPI decides the source

The rule: if the name is in the path, it is a **path parameter**; if its type is a Pydantic model, it is the **request body**; otherwise it is a **query parameter**. You almost never have to state it explicitly.

Seeing it work

Run the app and open `/docs`. The `POST /books` endpoint now shows a ready-to-edit JSON example built from your `BookIn` model, complete with field types. Edit it, execute, and watch validation in action when you remove a required field.

TRY IT YOURSELF — Feel the validation

POST a valid book and confirm it gets stored with a new id. Then POST `{{"title": "X", "author": "Y", "year": "soon"}}` — a string where an int belongs — and read the 422 telling you `year` must be an integer. You did not write a single line of that validation.

Pydantic Models

Pydantic is the validation engine beneath FastAPI. A Pydantic model is a Python class that describes the exact shape of your data — and then enforces it automatically. Master this and you have mastered the most important skill in the framework.

What a model is

You define a class that inherits from `BaseModel` and list its fields with type hints. That class becomes a strict contract: data that does not fit is rejected with a clear error, and data that fits arrives as a clean, typed Python object.

```
from pydantic import BaseModel

class Book(BaseModel):
    id: int
    title: str
    author: str
    year: int
    in_stock: bool = True           # default value -> optional field
```

A API TERM — Pydantic / BaseModel

Pydantic is the data-validation library FastAPI is built on. A class that inherits from **BaseModel** is a **model**: a typed, self-validating description of some data. Note we are on **Pydantic v2**, the current major version — its method names differ from old v1 tutorials.

Validation happens on creation

The moment you build a model, Pydantic checks and coerces every field:

```
Book(id=1, title="Dune", author="Frank Herbert", year=1965)
# -> a valid Book object

Book(id="not-a-number", title="X", author="Y", year=1965)
# -> raises ValueError: id must be an integer
```

Inside FastAPI you rarely call the constructor yourself — the framework does it for you when a request body matches a model, turning failures into HTTP `422` responses.

Rich field types

Pydantic understands far more than `str` and `int`. Using precise types makes your model both safer and self-documenting.


```

from datetime import date
from pydantic import BaseModel, EmailStr, HttpUrl

class Author(BaseModel):
    name: str
    email: EmailStr # must be a valid email address
    website: HttpUrl | None = None # must be a valid URL if present
    born: date # accepts "1920-10-08", gives a date object
    awards: list[str] = [] # a list of strings, defaults to empty
    rating: float | None = None

```

Type	Accepts	You get back
<code>str</code>	Text	str
<code>int / float</code>	Numbers (and numeric text)	int / float
<code>bool</code>	true/false, 1/0, yes/no	bool
<code>date / datetime</code>	ISO date strings	date / datetime object
<code>EmailStr</code>	Valid email text	validated str
<code>list[str]</code>	JSON array of text	Python list
<code>X None</code>	the value or nothing	X or None

NOTE — `EmailStr` needs one extra package

`EmailStr` relies on `email-validator`, which comes with `fastapi[standard]`. If you used a minimal install, run `pip install email-validator`.

Fine-grained rules with `Field()`

The `Field` function lets you attach constraints, defaults, and documentation to individual fields. Those constraints become validation *and* appear in your auto-generated docs.

```

from pydantic import BaseModel, Field

class Book(BaseModel):
    title: str = Field(min_length=1, max_length=120,
                      description="The book's title")
    author: str = Field(min_length=1, max_length=80)
    year: int = Field(ge=1450, le=2100,
                    description="Year of publication")
    pages: int = Field(gt=0, description="Must be positive")
    summary: str | None = Field(default=None, max_length=500)

```

Common `Field` constraints you will reach for constantly:

Constraint	Meaning
<code>min_length</code> / <code>max_length</code>	String length limits
<code>gt</code> / <code>ge</code>	Greater than / greater-or-equal (numbers)
<code>lt</code> / <code>le</code>	Less than / less-or-equal (numbers)
<code>default</code>	The value used when the field is omitted
<code>description</code>	Human text shown in the API docs
<code>pattern</code>	A regular expression the string must match

Nested models

Models can contain other models, mirroring real-world structure. FastAPI validates the whole tree and documents it beautifully.

```
class Publisher(BaseModel):
    name: str
    country: str

class Book(BaseModel):
    title: str
    year: int
    publisher: Publisher      # a model inside a model
    tags: list[str] = []
```

A client now sends nested JSON, and Pydantic validates every level:

```
{
  "title": "Dune",
  "year": 1965,
  "publisher": {"name": "Chilton Books", "country": "USA"},
  "tags": ["sci-fi", "classic"]
}
```

Separate input and output models — a professional habit

The data a client *sends* often differs from what you *store* or *return*. A client should not send an `id` (the server assigns it); you may not want to return internal fields. The clean solution is more than one model.

```
class BookCreate(BaseModel):      # what the client sends to create
    title: str = Field(min_length=1)
    author: str
    year: int = Field(ge=1450, le=2100)

class BookPublic(BaseModel):     # what the API returns
```

```
id: int
title: str
author: str
year: int
```

? WHY IT MATTERS — Two models prevent two classes of bugs

Input models stop clients from setting fields they shouldn't (like `id` or `is_admin`). Output models stop you from accidentally leaking internal or sensitive fields (like a password hash). This input/output split is standard practice in production FastAPI codebases — and it sets up Chapter 7 perfectly.

▶ TRY IT YOURSELF — Give BookShelf real validation

Replace the loose `BookIn` from Chapter 5 with a `BookCreate` that enforces a non-empty title and a sensible year range. In `/docs`, try to create a book with `year: 3000` and read the precise validation error.

Response Models & Status Codes

So far we have focused on what comes *in*. Now we take deliberate control of what goes *out*: the exact shape of the response and the status code that describes it.

Declaring a response model

You tell FastAPI the shape of a successful response with `response_model` in the decorator. FastAPI then filters your return value to exactly that shape, validates it, and documents it.

```
@app.post("/books", response_model=BookPublic)
def create_book(book: BookCreate):
    new = book.model_dump()
    new["id"] = len(books) + 1
    books.append(new)
    return new          # FastAPI shapes this to BookPublic on the way out
```

A API TERM — Response model

A Pydantic model that defines the structure of what your endpoint returns. FastAPI uses it to (1) drop any extra fields, (2) validate the output, and (3) document the response in `/docs`. It is the output counterpart to a request body model.

The killer feature: automatic field filtering

Imagine your stored user has a `hashed_password`. You return the whole object by mistake. With a response model that *omits* that field, FastAPI strips it out before it ever leaves the server — a security safety net.

```
class UserIn(BaseModel):
    username: str
    password: str          # client sends this

class UserOut(BaseModel):
    username: str          # note: NO password field

@app.post("/users", response_model=UserOut)
def create_user(user: UserIn):
    # even if we return the password, response_model removes it
    return user
```

? WHY IT MATTERS — Output filtering prevents data leaks

Forgetting to remove a sensitive field is one of the most common real-world API security bugs. A response model makes the safe path the default path: anything not declared simply cannot be returned.

Modern alternative: the return type annotation

In current FastAPI you can often skip `response_model=` and just annotate the function's return type. It does the same filtering and is cleaner.

```
@app.get("/books/{book_id}")
def get_book(book_id: int) -> BookPublic:
    ...

@app.get("/books")
def list_books() -> list[BookPublic]:    # a list of the model
    return books
```

Setting the status code

Successful responses default to `200 OK`. But creating a resource should return `201 Created`, and a delete with no body should return `204 No Content`. You set this with `status_code`, ideally via the readable constants in `fastapi.status`.

```
from fastapi import status

@app.post("/books", response_model=BookPublic,
          status_code=status.HTTP_201_CREATED)
def create_book(book: BookCreate):
    ...

@app.delete("/books/{book_id}",
            status_code=status.HTTP_204_NO_CONTENT)
def delete_book(book_id: int):
    ...
    return None
```

★ PRO TIP — Use the named constants

`status.HTTP_201_CREATED` reads better than a bare `201` and lets your editor autocomplete the right codes. It is the style the official docs use throughout.

Returning proper errors with HTTPException

Remember our clumsy `return {"error": "Book not found"}`? That returned status `200` — telling the client "success" while handing it an error. The correct tool is `HTTPException`, which sends a real error status.

```
from fastapi import HTTPException, status

@app.get("/books/{book_id}", response_model=BookPublic)
def get_book(book_id: int):
    for book in books:
```

```
    if book["id"] == book_id:
        return book
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"Book {book_id} not found",
    )
```

API TERM — HTTPException

A special exception you `raise` (not return) when something goes wrong. FastAPI catches it and produces a proper HTTP error response with your chosen status code and a JSON `detail` message. This is the standard way to signal 'not found,' 'forbidden,' and similar conditions.

WATCH OUT — raise it, don't return it

You `raise HTTPException(...)`. Returning it as a normal value will not work — FastAPI relies on the exception mechanism to interrupt the request and build the error response.

TRY IT YOURSELF — Make BookShelf honest

Add response models to all endpoints, set `201` on create and `204` on delete, and replace every fake-success error with a real `404` via `HTTPException`. Request a non-existent book and confirm the status code is now genuinely 404.

Forms & File Uploads

Not every client sends JSON. HTML forms and file uploads use a different encoding, and FastAPI handles both with dedicated tools. We will let users upload a cover image for a book.

Form data is not JSON

When a traditional HTML `<form>` submits, the browser does not send JSON — it sends `application/x-www-form-urlencoded` (or `multipart/form-data` when files are involved). To read those fields you use `Form` instead of a body model.

```
from typing import Annotated
from fastapi import Form

@app.post("/login")
def login(
    username: Annotated[str, Form()],
    password: Annotated[str, Form()],
):
    return {"username": username}
```

A API TERM — Form data vs JSON body

JSON is what programs and JavaScript apps usually send. **Form data** is what classic HTML forms send. They are different content types on the wire, so FastAPI gives you `Form()` to read form fields — you cannot mix a JSON body and `Form()` fields in the same endpoint.

i NOTE — Forms need python-multipart

Form and file support depends on the `python-multipart` package, which is included with `fastapi[standard]`. A minimal install needs `pip install python-multipart`.

Uploading files

For files, FastAPI gives you `UploadFile`. It exposes the filename, content type, and async methods to read the bytes — without loading huge files entirely into memory.

```
from fastapi import File, UploadFile

@app.post("/books/{book_id}/cover")
async def upload_cover(
    book_id: int,
    file: Annotated[UploadFile, File()],
):
    contents = await file.read()           # the raw bytes
```

```

return {
    "book_id": book_id,
    "filename": file.filename,
    "content_type": file.content_type,
    "size_bytes": len(contents),
}

```

A API TERM — UploadFile

FastAPI's representation of an uploaded file. Key attributes: `.filename`, `.content_type`, and the async methods `.read()`, `.write()`, `.seek()`, `.close()`. It streams from disk, so it handles large files gracefully.

Why the function is async here

Notice `async def` and `await file.read()`. `UploadFile`'s methods are asynchronous. You will learn async properly in Phase 2; for now, the rule is simple: when you call something with `await`, the function holding it must be `async def`.

Validating and saving the upload

In practice you check the file type and size, then save it. Here is a realistic version for BookShelf that rejects non-images and writes the cover to disk.

```

import shutil
from pathlib import Path
from fastapi import HTTPException, status

COVERS_DIR = Path("covers")
COVERS_DIR.mkdir(exist_ok=True)

@app.post("/books/{book_id}/cover")
async def upload_cover(
    book_id: int,
    file: Annotated[UploadFile, File()],
):
    if file.content_type not in {"image/jpeg", "image/png"}:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Only JPEG and PNG images are allowed",
        )
    dest = COVERS_DIR / f"{book_id}_{file.filename}"
    with dest.open("wb") as buffer:
        shutil.copyfileobj(file.file, buffer)  # stream to disk
    return {"book_id": book_id, "saved_as": str(dest)}

```


Combining form fields and files

A real upload form often carries metadata alongside the file. You can declare both in one endpoint, since both come from `multipart/form-data`.

```
@app.post("/books/{book_id}/cover")
async def upload_cover(
    book_id: int,
    caption: Annotated[str, Form()],
    file: Annotated[UploadFile, File()],
):
    return {"book_id": book_id, "caption": caption,
            "filename": file.filename}
```

★ PRO TIP — Multiple files

To accept several files at once, annotate as a list: `files: Annotated[list[UploadFile], File()]`. The Swagger UI will render a multi-file picker automatically.

▶ TRY IT YOURSELF — Add cover uploads to BookShelf

Add the validated `upload_cover` endpoint. In `/docs`, expand it, choose a real JPEG or PNG from your computer, and execute. Confirm the file appears in your `covers/` folder, then try a `.txt` file and watch the 400 rejection.

Automatic Docs: Swagger UI & ReDoc

One of FastAPI's standout gifts: it reads your code and builds complete, interactive API documentation for free. Two styles ship out of the box, both kept perfectly in sync with your endpoints because they are generated from the same source of truth.

Where the docs come from: OpenAPI

From your path operations, parameters, and Pydantic models, FastAPI generates a machine-readable description of your entire API following the **OpenAPI** standard. Both doc UIs simply render that description. You can view the raw document yourself at `/openapi.json`.

A API TERM — OpenAPI

An industry-standard, JSON/YAML format for describing what an HTTP API offers — its endpoints, inputs, outputs, and types. Because FastAPI emits standard OpenAPI, countless tools (doc renderers, client-code generators, testing tools) work with your API automatically. We use this for SDK generation in Phase 4.

Swagger UI — the interactive playground

Visit `http://127.0.0.1:8000/docs`. Swagger UI lists every endpoint, colour-coded by verb, and lets you *call them live*: expand an endpoint, click *Try it out*, fill in parameters or a body (pre-filled from your models), press *Execute*, and see the real request, response, and status code.

- Every path/query parameter is listed with its type and whether it is required.
- Request bodies show an editable JSON example generated from your Pydantic model.
- Response models and possible status codes are documented per endpoint.
- Validation constraints (min/max, lengths) appear right in the field descriptions.

? WHY IT MATTERS — Docs that cannot go stale

Hand-written API docs drift out of date the moment code changes. Because FastAPI generates docs from the same type hints that run your API, they are always exact. This single feature saves teams enormous amounts of time and is a major reason FastAPI gets adopted.

ReDoc — the clean reading view

Visit `http://127.0.0.1:8000/redoc`. ReDoc presents the same information in a calm, three-panel, document-style layout. It is not interactive, but it is excellent for sharing with stakeholders or reading reference material. Same data, different presentation.

	Swagger UI (/docs)	ReDoc (/redoc)
Best for	Trying endpoints live	Reading & sharing

	Swagger UI (/docs)	ReDoc (/redoc)
Interactive	Yes — execute requests	No
Layout	Expandable list	Three-panel document

Enriching the docs from your code

Everything you add to your code flows straight into the docs. Titles, descriptions, tags, and summaries all improve the generated pages.

```
app = FastAPI(
    title="BookShelf API",
    description="A personal library manager built while learning FastAPI.",
    version="1.0.0",
)

@app.get(
    "/books",
    tags=["Books"],
    summary="List all books",
    description="Returns every book, with optional pagination.",
)
def list_books(skip: int = 0, limit: int = 10):
    ...
```

`tags` group related endpoints into sections; the function's docstring also becomes the long description if you prefer writing it there. Pydantic `Field(description=...)` text shows up next to each field.

★ PRO TIP — Adjust or disable the docs

You can move the docs (`FastAPI(docs_url="/playground")`) or disable them in production (`docs_url=None`). Locking docs behind authentication is a common professional choice we revisit in the security chapters.

▶ TRY IT YOURSELF — Polish your API's front page

Add a `title`, `description` and `version` to your `FastAPI()`, then tag your endpoints with `tags=["Books"]`. Reload `/docs` and `/redoc` and admire the professional documentation you just produced without writing any docs.

The Complete BookShelf API

Here is everything from Phase 1 assembled into one coherent, runnable application. Every concept you learned is present: typed path and query parameters, validated request bodies, separate input/output Pydantic models, proper response models and status codes, real error handling, and a file upload — all self-documenting.

main.py – the complete Phase 1 BookShelf API

```
from pathlib import Path
import shutil
from typing import Annotated

from fastapi import (
    FastAPI, HTTPException, Query, Form, File, UploadFile, status,
)
from pydantic import BaseModel, Field

app = FastAPI(
    title="BookShelf API",
    description="A personal library manager built while learning FastAPI (Phase 1).",
    version="1.0.0",
)

# ---- Models -----
class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=120)
    author: str = Field(min_length=1, max_length=80)
    year: int = Field(ge=1450, le=2100)
    in_stock: bool = True

class BookPublic(BaseModel):
    id: int
    title: str
    author: str
    year: int
    in_stock: bool

# ---- In-memory "database" -----
books: list[dict] = [
    {"id": 1, "title": "Dune", "author": "Frank Herbert",
     "year": 1965, "in_stock": True},
    {"id": 2, "title": "1984", "author": "George Orwell",
     "year": 1949, "in_stock": False},
]

def _find(book_id: int) -> dict | None:
    return next((b for b in books if b["id"] == book_id), None)
```

```

# ---- Endpoints -----
@app.get("/books", response_model=list[BookPublic], tags=["Books"],
        summary="List books")
def list_books(
    skip: int = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 10,
    author: str | None = None,
):
    results = books
    if author:
        results = [b for b in results if b["author"] == author]
    return results[skip : skip + limit]

@app.get("/books/search", response_model=list[BookPublic], tags=["Books"])
def search_books(q: Annotated[str, Query(min_length=2, max_length=50)]):
    return [b for b in books if q.lower() in b["title"].lower()]

@app.get("/books/{book_id}", response_model=BookPublic, tags=["Books"])
def get_book(book_id: int):
    book = _find(book_id)
    if book is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND,
                            detail=f"Book {book_id} not found")
    return book

@app.post("/books", response_model=BookPublic, tags=["Books"],
         status_code=status.HTTP_201_CREATED)
def create_book(book: BookCreate):
    new = book.model_dump()
    new["id"] = (max((b["id"] for b in books), default=0)) + 1
    books.append(new)
    return new

@app.put("/books/{book_id}", response_model=BookPublic, tags=["Books"])
def update_book(book_id: int, book: BookCreate):
    existing = _find(book_id)
    if existing is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND,
                            detail=f"Book {book_id} not found")
    existing.update(book.model_dump())
    return existing

@app.delete("/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT,
           tags=["Books"])
def delete_book(book_id: int):
    book = _find(book_id)
    if book is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND,
                            detail=f"Book {book_id} not found")
    books.remove(book)
    return None

```

```
COVERS = Path("covers"); COVERS.mkdir(exist_ok=True)

@app.post("/books/{book_id}/cover", tags=["Covers"])
async def upload_cover(
    book_id: int,
    caption: Annotated[str, Form()],
    file: Annotated[UploadFile, File()],
):
    if _find(book_id) is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, detail="Book not found")
    if file.content_type not in {"image/jpeg", "image/png"}:
        raise HTTPException(status.HTTP_400_BAD_REQUEST,
                             detail="Only JPEG/PNG allowed")
    dest = COVERS / f"{book_id}_{file.filename}"
    with dest.open("wb") as out:
        shutil.copyfileobj(file.file, out)
    return {"book_id": book_id, "caption": caption, "saved_as": str(dest)}
```

★ PRO TIP — Run your capstone

Save this as `main.py`, run `fastapi dev main.py`, and open `/docs`. You now have a complete, validated, documented CRUD API with file upload — built entirely from Phase 1 concepts.

What you can now do

- ✓ Set up an isolated environment and install FastAPI the recommended way
- ✓ Design RESTful endpoints with the four core HTTP verbs
- ✓ Capture and validate data from the path, the query string, and the request body
- ✓ Model and validate data with Pydantic v2, including fields, constraints, and nesting
- ✓ Control exactly what your API returns and with which status code
- ✓ Return correct errors with HTTPException instead of fake successes
- ✓ Accept form submissions and validated file uploads
- ✓ Read and enrich the automatic Swagger UI and ReDoc documentation

Glossary of API Terms

Every term introduced in Phase 1, in one place, for quick review.

Term	Meaning
API	A defined way for one program to request data or actions from another.
ASGI	The asynchronous standard interface between a server (Uvicorn) and an async framework (FastAPI).
Annotated	Standard Python syntax for attaching metadata (like validation rules) to a type hint.
Client / Server	The client sends requests; the server answers them. Your FastAPI app is the server.
CRUD	Create, Read, Update, Delete — the four basic operations on stored data.
Decorator	An <code>@</code> -line above a function that adds behaviour; here, it registers an endpoint.
Endpoint	A verb + path your API responds to. Also called a route or path operation.
Form data	The encoding HTML forms use to submit, distinct from a JSON body; read with <code>Form()</code> .
HTTP	The protocol carrying requests and responses between client and server.
HTTPException	An exception you raise to return a proper HTTP error status and message.
Idempotency	A property where repeating an operation has the same effect as doing it once (PUT, DELETE).
JSON	A text format for structured data; looks like a Python dict and is the default API payload.
OpenAPI	The standard description format FastAPI auto-generates; powers the docs and tooling.
Pagination	Returning a large collection in small chunks, commonly via skip/limit.
Path parameter	A required value embedded in the URL path, identifying a resource (<code>/books/5</code>).
Pydantic / BaseModel	The validation library FastAPI uses; models inherit from <code>BaseModel</code> (v2).
Query parameter	An optional named input after <code>?</code> in the URL, used to filter or modify.
Request body	Structured data (usually JSON) a client sends, typically with POST or PUT.
Response model	A model defining the shape of what an endpoint returns; filters and documents output.
REST	A design style treating data as resources addressed by URLs and acted on with HTTP verbs.
Status code	A 3-digit number summarising the result: 2xx success, 4xx client error, 5xx server error.

Term	Meaning
UploadFile	FastAPI's object for an uploaded file, exposing filename, content type, and read methods.
Uvicorn	The high-performance ASGI server that actually runs your FastAPI application.
Virtual environment	An isolated, per-project Python package folder, created with <code>python -m venv</code> .

What Comes Next

You have gone from never having seen an API to building a complete, validated, self-documenting one. That is the entire foundation. Everything in the later phases builds directly on what you now understand.

COMING IN PHASE 2 · INTERMEDIATE

A real, database-backed API

The next project graduates from an in-memory list to a real database with proper structure, authentication, and production patterns.

Phase 2 topics

- **Dependency Injection** (`Depends`) — share and reuse logic cleanly
- **Routers & project structure** (`APIRouter`) — split a growing app into files
- **Async / await in FastAPI** — the concurrency model, properly explained
- **Databases with SQLAlchemy** (sync) and the **async** approaches
- **Alembic migrations** — evolving your database schema safely
- **Authentication** — OAuth2 & JWT, the standard way to secure an API
- **Middleware** (CORS & custom), **error handling**, and **background tasks**

Before you move on — solidify Phase 1

- ✓ Rebuild the BookShelf API from memory, without looking, to prove the concepts stuck
- ✓ Add a `PATCH` endpoint for partial updates and notice how it differs from PUT
- ✓ Add richer validation: a non-empty author, an ISBN field with a regex pattern
- ✓ Explore every corner of your `/docs` page and read the generated `/openapi.json`

NOTE — You are now an intermediate FastAPI developer

The request → function → response loop, type-driven validation, Pydantic modeling, and automatic documentation are the pillars the entire framework rests on. Carry them into Phase 2 and the advanced material will feel like a natural extension rather than a new language.

End of Phase 1. This guide was written and verified against the official FastAPI documentation at fastapi.tiangolo.com — 2026 edition, FastAPI 0.132+, Python 3.10+, Pydantic v2.